Name: Bazila Sulthana
RollNo: 2545
Date: 22/10/2026

ML project: Sales prediction

A Comparative Study of Linear Regression and Gradient Boosting Regression

1. Aim

The aim of this project is to build and evaluate machine learning regression models that estimate sales using the available features in a public Superstore sales dataset. Linear Regression and Gradient Boosting Regression are trained and compared using standard regression evaluation metrics.

2. Software and Tools Required

- Python 3
- Google Colab or Jupyter Notebook
- pandas and NumPy for data handling
- scikit-learn for preprocessing, model training, and evaluation
- Kaggle Superstore CSV dataset

3. Introduction

Sales prediction is the process of estimating future or expected sales from historical sales-related information. Reliable sales estimates can support inventory planning, budgeting, staffing, and business decisions. Machine learning can learn patterns from historical records and use them to estimate a numerical target such as Sales.

This project uses a supervised learning approach. The dataset contains examples with input features and a known Sales value. The model learns from a training portion of the data and is evaluated on a separate test portion to estimate how well it generalises to unseen records.

The main aim of this project is to develop a sales prediction model using Python and machine learning techniques. The project uses a public Superstore sales dataset obtained from Kaggle. The dataset contains sales-related information that can be used to train and evaluate regression models.

4. Problem Statement and Objectives

4.1 Problem Statement

Sales prediction is an important task for businesses because it helps them understand their expected sales and make better business decisions. Sales depend on various factors, such as shipping mode, customer segment, geographical information, product category, sub-category, quantity, and

discount. Understanding the relationship between these factors and the final sales value can help businesses plan their activities more effectively.

Estimating sales manually for a large number of orders is a difficult and time-consuming process. Manual estimation may also lead to errors because it is difficult to identify all the patterns and relationships present in a large dataset. Therefore, an automated method is useful for analysing historical sales records and predicting sales values.

This project develops a machine learning-based sales prediction workflow using the Superstore dataset. It uses regression algorithms to predict the numerical Sales value based on the available input features. Linear Regression and Gradient Boosting Regression are selected for model development and comparison.

The dataset is divided into training and testing sets. The training data is used to fit the models, while the testing data is used to evaluate their performance on unseen records. Performance is measured using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the R^2 score.

The main problem addressed by this project is to determine how effectively the two selected machine learning algorithms predict sales from the available dataset features and to identify which model performs better on the test data.

4.2 Objectives

The main objective of this project is to develop and evaluate machine learning models that predict sales values using Superstore data. The specific objectives are:

1. To load and inspect the dataset: Import the Superstore dataset into Python and examine its structure, number of records, columns, and data types.
2. To identify the target variable and input features: Select Sales as the target variable and identify the relevant input features used by the models.
3. To preprocess the data: Convert the target to numeric form, remove records with missing target values if present, and prepare numerical and categorical features for model training.
4. To split the dataset: Divide the prepared dataset into training and testing sets so that the models can be evaluated on unseen records.
5. To train Linear Regression: Develop a Linear Regression model as a baseline for comparison.
6. To train Gradient Boosting Regression: Build a Gradient Boosting Regression model using the selected parameters.
7. To evaluate model performance: Calculate MAE, MSE, RMSE, and R^2 score for both models.
8. To compare the models: Compare the evaluation results to determine which model performs better on the test dataset.
9. To inspect predictions: Display sample actual and predicted sales values from the selected model.

5. Dataset Description

The dataset used in this project is a public Superstore dataset obtained from Kaggle. It contains historical sales-related information that can be used to develop machine learning models for sales

prediction. The version loaded in the Google Colab notebook contains 9,994 records and 13 columns. Each record represents a sales transaction line.

The columns shown in the notebook are Ship Mode, Segment, Country, City, State, Postal Code, Region, Category, Sub-Category, Sales, Quantity, Discount, and Profit. The dataset contains numerical and categorical information.

The notebook output shows zero missing values in each of the 13 columns when the dataset is first loaded. The target-cleaning step then converts Sales to numeric form and removes rows with missing Sales values, if any.

5.1 Dataset Features

The dataset includes different types of attributes that describe sales transactions. The features used by the model are:

- Shipping information: Ship Mode.
- Customer segment information: Segment.
- Geographical information: Country, City, State, Postal Code, and Region.
- Product information: Category and Sub-Category.
- Numerical transaction information: Quantity and Discount.

Sales is the target variable. Profit is excluded from the input features. The notebook reports 11 input features before categorical encoding.

5.2 Target Variable

The target variable selected for this project is Sales. It represents the sales amount associated with a record in the dataset. Since Sales is a numerical value, the problem is treated as a regression task. The machine learning models learn from the input features in the training dataset and use the learned relationships to predict Sales values for records in the testing dataset.

5.3 Input Features

Input features are the variables provided to the machine learning models to predict the target variable. The code selects the available columns other than Sales and excludes identifier or name-like columns when they are present. The Profit column is also excluded because it is calculated separately from Sales and may introduce target-related information.

For the dataset used in this notebook, the resulting input features are Ship Mode, Segment, Country, City, State, Postal Code, Region, Category, Sub-Category, Quantity, and Discount. Categorical features are converted into numerical form using one-hot encoding, while numerical features are passed through numerical preprocessing.

5.4 Dataset Preparation

The dataset is uploaded and read into a Pandas DataFrame in Google Colab using the latin1 encoding. The code checks the dataset shape, displays the first five rows, lists the column names, and prints the missing-value count for each column.

The notebook reports 9,994 total records, 7,995 training records, 1,999 testing records, and 11 input features. The data is divided using an 80:20 train-test split with random_state=42.

6. Data Preprocessing

Data preprocessing is an important step in a machine learning project. Raw datasets may contain different data types and categorical information that cannot be directly used by regression algorithms. Therefore, the dataset is prepared before training the machine learning models.

In this project, the Superstore dataset is processed using Python, Pandas, and scikit-learn in Google Colab. The preprocessing steps implemented in the notebook are described below.

6.1 Loading the Dataset

The first step is to upload and read the Superstore CSV file into a Pandas DataFrame. The dataset is loaded using pd.read_csv() with latin1 encoding. After loading the file, the code displays the dataset shape, first five rows, column names, and missing-value counts.

6.2 Handling Missing Target Values

The Sales column is identified as the target variable. The code converts Sales into numeric form using pd.to_numeric() with errors="coerce". Rows with missing or non-numeric Sales values are then removed because the model requires a known numerical target during training.

6.3 Selecting Input Features

After identifying the target variable, the input features are selected from the remaining dataset columns. The code excludes identifier and name-like columns such as Row ID, Order ID, Customer ID, Customer Name, Product ID, and Product Name when present. The Profit column is also excluded from the input features.

6.4 Separating Numerical and Categorical Features

The dataset contains both numerical and categorical columns. The code identifies numerical columns using their data types and treats the remaining columns as categorical features. This separation allows a suitable preprocessing method to be applied to each group.

- Numerical features: These contain numerical values, such as Postal Code, Quantity, and Discount.
- Categorical features: These contain labels or categories, such as Ship Mode, Segment, City, State, Region, Category, and Sub-Category.

6.5 Handling Missing Values

Missing values in numerical and categorical input features are handled separately using scikit-learn SimpleImputer within preprocessing pipelines.

For numerical columns, missing values are filled using the median. For categorical columns, missing values are filled using the most frequent value, also called the mode. This prepares the input features for model training without unnecessarily removing records because of missing predictor values.

6.6 Encoding Categorical Variables

Machine learning algorithms generally require input features in numerical form. However, categorical columns contain text labels or categories. Therefore, OneHotEncoder is applied to

convert categorical variables into numerical columns. The encoder uses handle_unknown="ignore" so that categories not seen during training do not cause an error during transformation.

6.7 Splitting the Dataset

After selecting the input features and target, the dataset is divided into training and testing sets using an 80:20 split. The training set contains 7,995 records, while the testing set contains 1,999 records. The training set is used to fit the Linear Regression and Gradient Boosting Regression models. The testing set is used to evaluate their performance on records not used for training.

The parameter random_state=42 is used to make the random split reproducible. The notebook reports 11 input features before the preprocessing pipeline expands categorical variables through one-hot encoding.

7. Algorithm / Model

Linear Regression

Linear Regression is a supervised learning algorithm used to predict a numerical value by learning a linear relationship between input features and the target variable. It predicts the target as a weighted sum of the input features:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n$$

The coefficients are calculated using the Ordinary Least Squares method, which finds values that minimise the sum of squared differences between actual and predicted values. In this project, Linear Regression is used as the baseline model.

One advantage of Linear Regression is that it is fast and easy to interpret. However, it may not capture complex non-linear relationships between features.

Gradient Boosting Regression

Gradient Boosting Regression is an ensemble method that builds a strong model from multiple small decision trees. It starts with an initial prediction and adds trees sequentially. Each new tree learns from the errors made by the model built so far, and its output is added with a learning-rate weight.

The GradientBoostingRegressor in this project is configured with n_estimators=100, learning_rate=0.1, max_depth=3, and random_state=42. These are the parameter values used in the notebook.

Gradient Boosting can learn non-linear relationships and interactions between features. However, its performance depends on the selected parameters, and further tuning may be useful.

8. Results and Model Comparison

After training the Linear Regression and Gradient Boosting Regression models, both models were evaluated using the testing dataset. Four evaluation metrics were used to compare their performance: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score. The results below are the values printed by the Google Colab notebook.

Evaluation Metric	Linear Regression	Gradient Boosting Regression
MAE	242.4177	200.5197
MSE	486525.3473	470681.8153
RMSE	697.5137	686.0625
R^2 Score	0.1764	0.2032

8.1 Comparison and Discussion

The results show that Gradient Boosting Regression performed better than Linear Regression across all four evaluation metrics in this experiment.

- MAE: Gradient Boosting Regression produced a lower error of 200.5197 compared with 242.4177 for Linear Regression.
- MSE: Gradient Boosting Regression achieved a lower value of 470681.8153 compared with 486525.3473.
- RMSE: Gradient Boosting Regression obtained a lower value of 686.0625 compared with 697.5137.
- R^2 Score: Gradient Boosting Regression achieved a higher score of 0.2032 compared with 0.1764.

Lower error values indicate smaller prediction errors, while a higher R^2 score indicates better performance relative to the mean-value baseline. Therefore, Gradient Boosting Regression is the better-performing model among the two models tested in this experiment. The notebook also selects Gradient Boosting Regression as the model with the lowest RMSE.

However, the R^2 score of 0.2032 indicates that the model explains only a limited proportion of the variation in Sales. Therefore, the results do not indicate highly accurate sales prediction, and further improvements may be required.

9. Results and Discussion

The performance of Linear Regression and Gradient Boosting Regression was evaluated using the testing dataset. Both models completed the sales prediction task, but their evaluation results showed differences in prediction performance.

The Linear Regression model achieved an MAE of 242.4177, an MSE of 486525.3473, an RMSE of 697.5137, and an R^2 score of 0.1764. These results indicate that the model can estimate sales values, but its ability to explain variations in sales is limited.

The Gradient Boosting Regression model achieved an MAE of 200.5197, an MSE of 470681.8153, an RMSE of 686.0625, and an R^2 score of 0.2032. Compared with Linear Regression, it produced lower prediction errors and a higher R^2 score. Therefore, Gradient Boosting Regression performed better across all four evaluation metrics in this experiment.

The improved performance of Gradient Boosting Regression suggests that the dataset may contain relationships that are better represented by the tree-based ensemble model than by a simple linear model. However, the R^2 score of 0.2032 is relatively low. This indicates that the model explains

only a limited proportion of the variation in Sales. Therefore, although Gradient Boosting Regression performed better than Linear Regression, its predictions still have considerable limitations.

The prediction performance may be improved through further feature engineering, hyperparameter tuning, and validation using different data splits. Additional experiments with other regression algorithms may also help identify a more suitable model.

Overall, the results demonstrate that machine learning algorithms can be applied to sales prediction and that comparing different models helps identify their relative strengths and weaknesses. Further testing and improvement are necessary before using the model to make important real-world business decisions.

10. Conclusion

This project developed a machine learning-based sales prediction workflow using a public Superstore dataset obtained from Kaggle. The main aim was to predict sales values using historical sales-related information and compare the performance of two regression algorithms: Linear Regression and Gradient Boosting Regression.

The project involved dataset loading, data preprocessing, model training, and model evaluation. During preprocessing, the Sales target was converted to numeric form, identifier and name-like columns and Profit were excluded from the predictors, numerical and categorical features were identified, missing input values were handled using median and most-frequent-value imputation, and categorical variables were converted into numerical representations using one-hot encoding. The prepared dataset was divided into training and testing sets.

Both regression models were trained and evaluated using the same testing dataset. Their performance was compared using MAE, MSE, RMSE, and the R^2 score. The results showed that Gradient Boosting Regression performed better than Linear Regression across all four evaluation metrics. It achieved an MAE of 200.5197 and an R^2 score of 0.2032, while Linear Regression achieved an MAE of 242.4177 and an R^2 score of 0.1764.

However, the R^2 score of Gradient Boosting Regression indicates that the model explains only a limited proportion of the variation in Sales. Therefore, the overall predictive performance is modest, and the model should not be considered highly accurate in its current form.

The project demonstrates how machine learning techniques can be applied to historical sales data and how different regression algorithms can be compared to identify their relative performance. It also highlights the importance of data preprocessing, appropriate feature selection, and model evaluation in developing a prediction workflow.

In conclusion, Gradient Boosting Regression was the better-performing model among the two algorithms tested. Although the current results show an improvement over Linear Regression, further development and validation are required to achieve more reliable sales predictions.

11. Future Scope

The current Sales Prediction project demonstrates how machine learning algorithms can be used to predict sales values from historical sales-related data. Although Gradient Boosting Regression performed better than Linear Regression, the overall predictive performance is still limited. Therefore, several improvements can be explored in future work.

11.1 Hyperparameter Tuning

The performance of Gradient Boosting Regression depends on parameters such as the learning rate, number of estimators, and maximum tree depth. In the future, these parameters can be adjusted using hyperparameter tuning techniques and cross-validation. This may help identify a better combination of parameters and improve prediction performance.

11.2 Feature Engineering and Selection

Feature engineering can be used to create useful new features from the available dataset attributes. Feature selection can also help identify useful input variables and remove irrelevant or redundant features. These improvements may help the models learn more meaningful relationships in the data.

11.3 Time-Based Validation

The current notebook uses a random 80:20 train-test split. If the project is later extended to forecast sales for future periods using a dataset containing suitable time information, time-aware validation can be used instead. Historical records can be used for training, while later periods are reserved for testing. This approach can provide a more realistic evaluation for future-period forecasting.

Overall, these future improvements may help develop a more reliable and practical sales prediction system. Further experimentation, validation, and testing are necessary before the model can be used for real-world business decisions.

12. References

10. Kaggle, public Superstore sales dataset used for this project.
11. scikit-learn documentation: LinearRegression, GradientBoostingRegressor, model_selection, preprocessing, and regression metrics.
12. pandas documentation for reading and preparing CSV data.
13. Google Colab notebook used to run the project code.